Discrete Response, Time Series and Panel Data - Notes

Nils Rechberger

2026-02-26

W-01-Timeseries-Introduction-Visualisation-Stationarity

> **Note**
>
> Coming soon...

W-02-Timeseries-Introduction-Visualisation-Stationarity

Log-transformations

Instead of the original time series , consider the log-transformed time series with the elements.

$$g_t = \log(X_t)$$

When to apply it?:

- When the distribution of the time series elements is right-skewed.
- When a time series could be produced by the product of model outcomes.

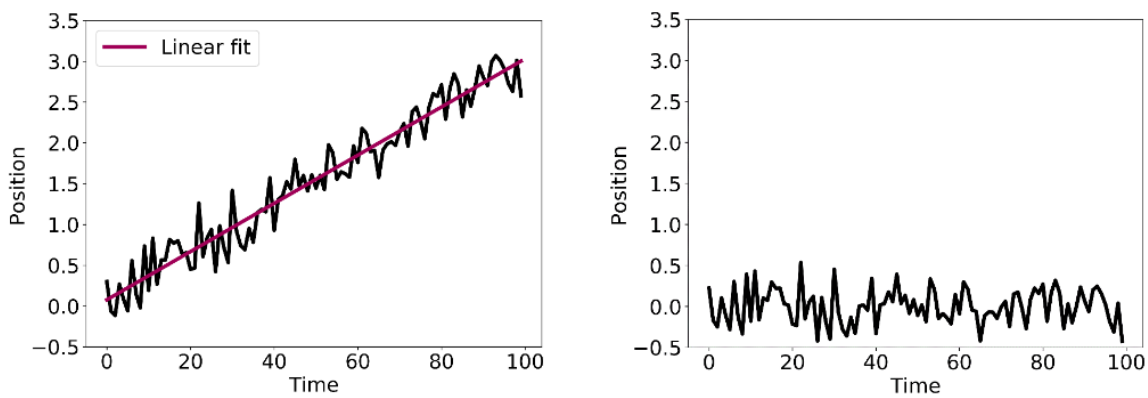
Differencing

Let's take a linear function (defined on a set of integer numbers)

$$X_t = m \cdot t + b$$

Then, the backward difference

$$X_t - X_{t-1} = (m \cdot t + b) - (m \cdot (t-1) + b) = m$$



$$\boldsymbol{X_t = 0.03 \cdot t + U_t}$$
$$U_t \sim \mathcal{N}(0,0.3)$$
Linear fit on X_t: $0.02596 \cdot t + 0.0759$

$$\boldsymbol{Y_t = X_t - (0.02596 \cdot t + 0.0759)}$$

Figure 1: Differencing

Higher Order Differencing

Use multiple derivatives to eliminate lower-order terms. Trend order is reduced, but the stationary part may be more complex. After trend is eliminated, maybe a new dependencies in the stationary part.

Example

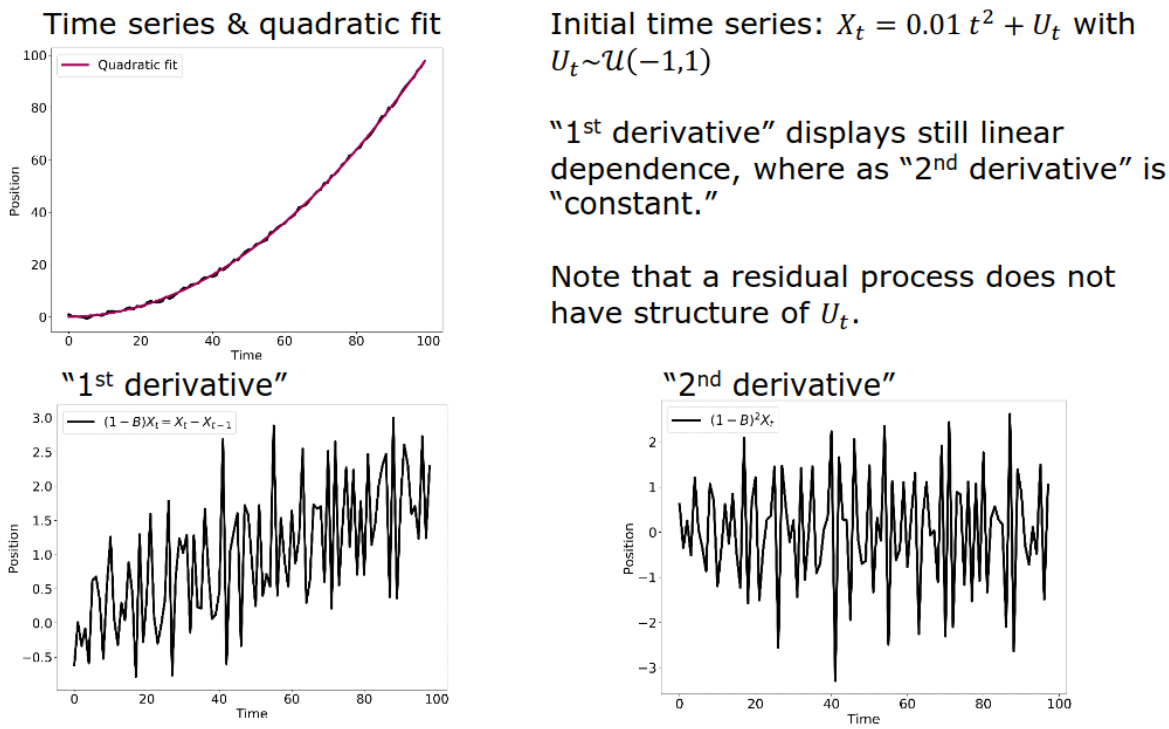
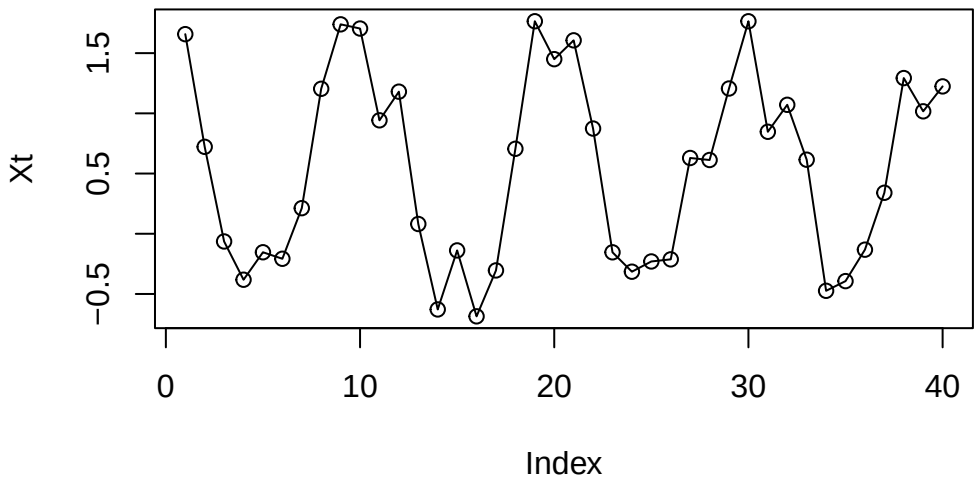


Figure 2: Remove Higher Order Trend

Diferencing Seasonality

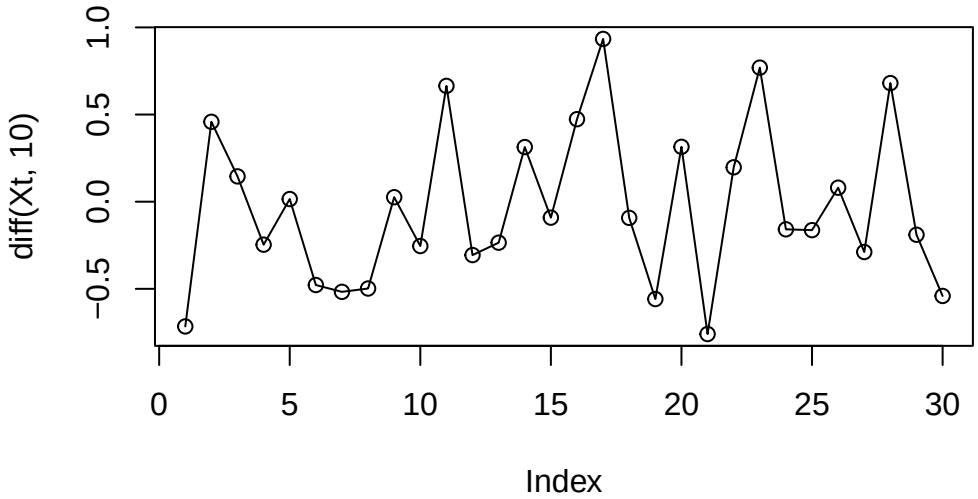
Seasonality manifests itself by a repetitive behavior after a fixed lag time p.

```
tRange<-1:40
Xt<-cos(2*pi/10*tRange)+runif(40)
plot(Xt,type="o")
```



What happens if we differentiate now with lag 10?

```
plot(diff(Xt,10),type="o")
```



Summary of differencing

Advantages:

- can remove trends and seasonality effects
- is easy to use

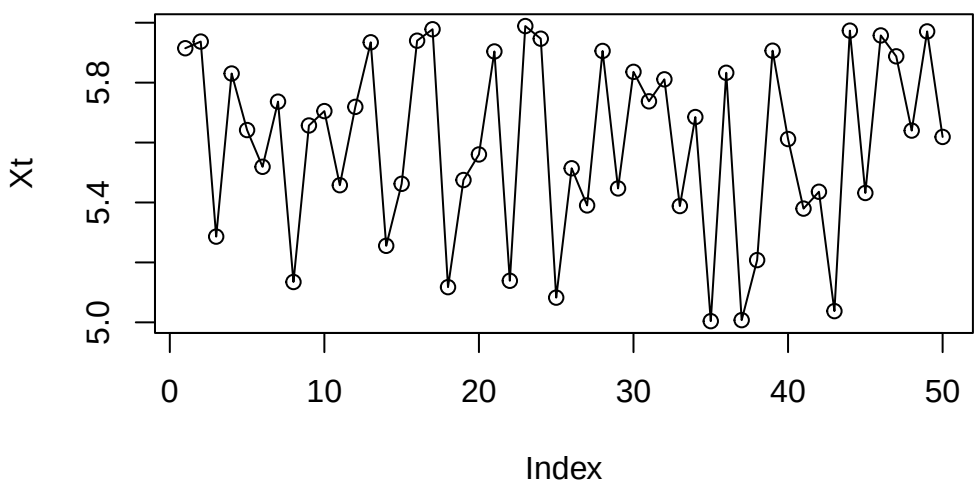
Disadvantages:

- the resulting time series is shorter than the original one
- the results of differencing may display dependencies among different elements
- the original decomposition elements are unknown

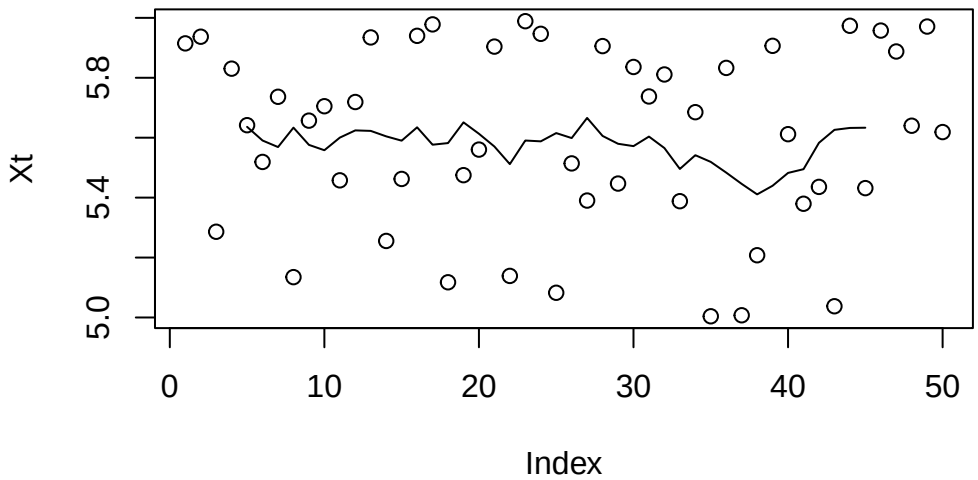
Filtration

To reduce the noise element in an experiment, you can repeat the measurement multiple times and then calculate the average.

```
set.seed(42)
Xt<-5+runif(50)
plot(Xt,type="o")
```



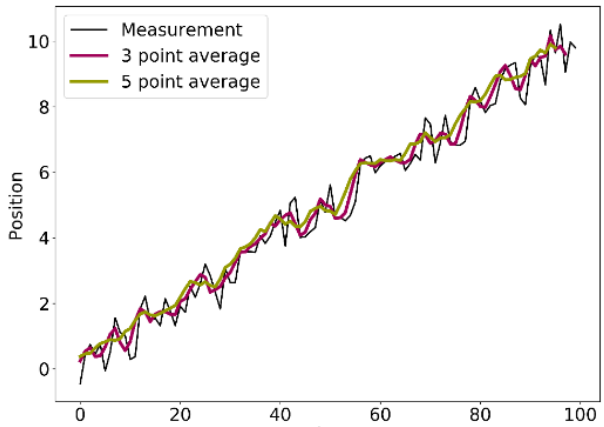
```r
filtered<-filter(Xt,filter=rep(1,10)/10)
plot(Xt)
points(filtered,type="l")
```



Reasons for using Filtration

By averaging measurements, we obtain a new averaged time series. By increasing the number of averaged measurements, the parts converges to the expectation value of the measurement error, i.e. 0.

Note: The short-term variations are smoothed out and the trend is confirmed.



Initial time series:
$X_t = 0.1\,t + U_t, U_t \sim \mathcal{U}(-1,1)$

3 point average: $a_0 = a_{-1} = a_{-2} = \frac{1}{3}$

5 point average: $a_0 = a_{-1} = a_{-2} = a_{-3} = a_{-4} = \frac{1}{5}$

Figure 3: Filtering

Linear filters to quantify seasonality effects

In case of seasonality effects, the filter has to expand over the full period to extract the trend. To quantify the seasonal impact, the values of the same time in the season have to be averaged.

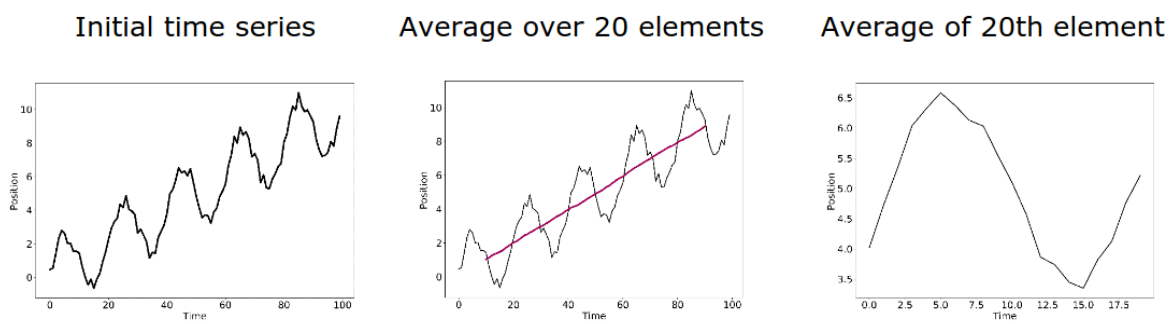


Figure 4: Quantify Seasonality Effects

Seasonal-trend decomposition by Loess Algorithm (STL)

Linear filters are great if periodicity is obvious; but in less obvious cases, extracting the individual components may be laborious. An alternative is the seasonal-trend decomposition procedure by Loess: An iterative, non-parametric smoothing algorithm that is more robust.

Fitting of Periodic Function

If you know the generative model, you can use fitting to determine the contributions of trend and seasonality. Fitting methods (among others):

- Ordinary least square (OLS)
- Robust fitting (to prevent the impact of outliers)
- GLS (generalised least square) / time series regression methods

```
set.seed(42)
#Generate the sample data set
t<-seq(1,100,length=100)
data<-0.1*t+cos(2*pi/10*t)+runif(100)
ts<-ts(data)
#Fit the model
fit<-lm(ts~t+cos(2*pi/10*t))
summary(fit)
```

```
Call:
lm(formula = ts ~ t + cos(2 * pi/10 * t))

Residuals:
     Min       1Q   Median       3Q      Max 
-0.58266 -0.22271  0.04478  0.25411  0.49285 

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)    
(Intercept)        0.639744   0.059840   10.69   <2e-16 ***
t                  0.097718   0.001029   94.98   <2e-16 ***
cos(2 * pi/10 * t) 0.973247   0.041999   23.17   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2969 on 97 degrees of freedom
Multiple R-squared:  0.9901,	Adjusted R-squared:  0.9899 
F-statistic:  4836 on 2 and 97 DF,  p-value: < 2.2e-16
```

Autocorrelation

The autocorrelation depends only on the lag for weakly or strongly stationary time series.

$$p(k) := Cor(X_{t+k}, X_t)$$

Interpretation of autocorrelation

The correlation $p(k)^2$ squared corresponds to the percentage of variability explained by the linear association between X_t and X_{t+k}.

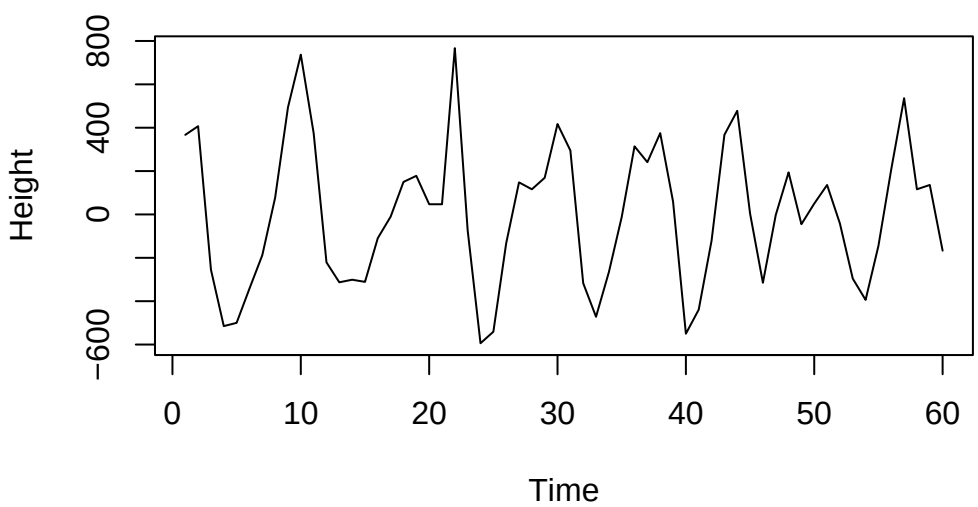
Lagged Scatter Plot

Exchange average over realizations with average over time.

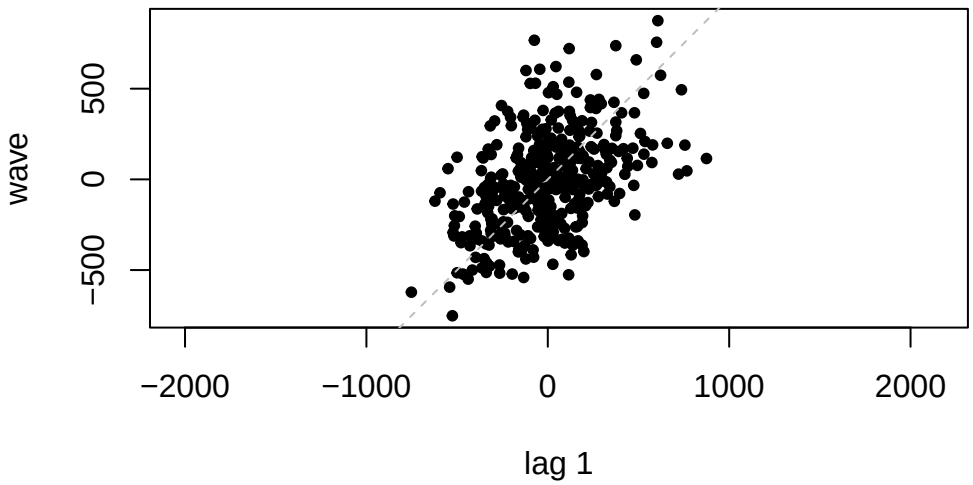
```
address <- "https://raw.githubusercontent.com/AtefOuni/ts/master/Data/wave.dat"
dat <- read.table(address,header=T)

#Generate ts object
wave<-ts(dat$waveht)

#Visualize the data
plot(window(wave,1,60),ylab="Height")
```



```r
#Lag plot
lag.plot(wave,do.lines=FALSE,pch=20)
```

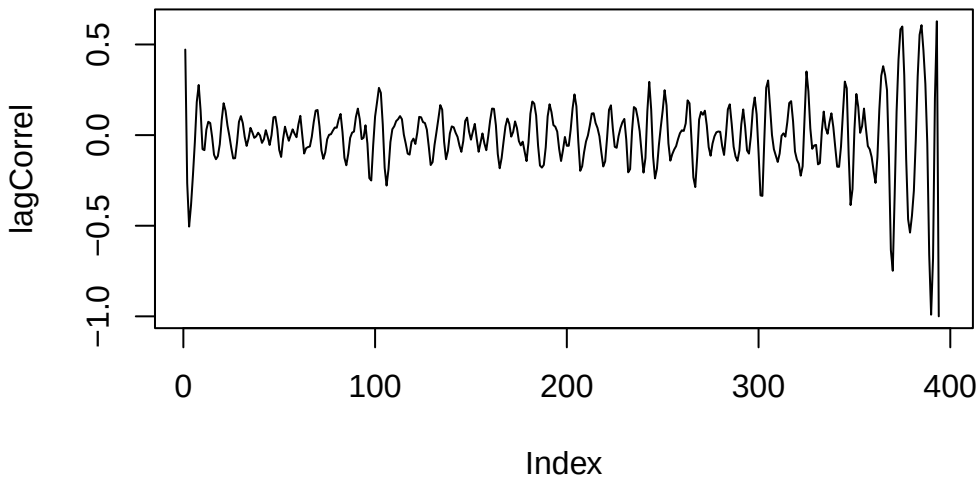


Calculate lagged scatter plot estimator

```r
# Calculate the Pearson Correlation coefficients
n <- length(wave)
lagCorrel <- rep(0,n)

for (i in 1:(n-1)){
lagCorrel[i]=cor(wave[1:(n-i)],wave[(i+1):n])
}

plot(lagCorrel,type="l")
```



Plug-In Estimation

The plug-in estimator is the standard approach to computation autocorrelations. Calculated with an `acf` function in R.

Summary Autocorrelation

- (Auto)correlation measures the linear association at a given lag
- For realizations, the autocorrelation is estimated by running the calculations over time instances instead of realizations (therefore stationarity is desired)
  - Lagged scatter plot estimator
  - Plug-in estimator
- For high lags, the lagged scatter plot estimator diverges (because too few time points are considered)
- The plug-in estimator corrects the diverging behavior but generates a bias.

> ⚠ Warning
>
> Warning: Non-linear relations may confuse the autocorrelation.